



ON-AIR GRAPHICS – THE BUG, THE WEATHER DESK, THE CRAWL

DOC NO. RL-Z0-WEATHER-A REV. A RIPOSTE LABORATORIES

Everything the channel puts on top of the picture.
Four overlays and one full-screen segment, all fed
by one script on a timer; nothing here needs a hu-
man once it is running.

CONTENTS

► Requires ErsatzTV ≥ 25.5.0	3
► The weather desk	3
A music bed	4
► Things that will bite you	4
► Changing the channel's resolution	7
► Wiring it into the schedule	7
► Performance	9

WHAT	WHERE IT COMES FROM	REFRESH
Channel bug (bottom right)	branding/z0-bug.png, made by tools/make-bug.sh	static
Weather card (top right)	branding/z0-weather-card.png	every 15 min
Info strip (bottom)	branding/z0-crawl.ass – forecast + the next programmes off the EPG, paged not scrolled	every 15 min
"NEXT ." lower third	ErsatzTV's own guide, via epg_entries	per programme
The Weather Desk (full screen)	weather/z0-local-forecast.mp4, 2 min, 4×/day	hourly

SEC.01

REQUIRES ERSATZTV ≥ 25.5.0

The graphics engine does not exist before that. On 25.2 the only overlay available is a single static `ChannelWatermark` image – no text, no templating, no crawl. The station runs **26.7.1** (`ghcr.io/ersatztv/legacy`).

Note the image moved: `ghcr.io/ersatztv/ersatztv` published no - `nvidia` build after v25.2.0 and the line now lives at `ghcr.io/ersatztv/legacy`, one unified image containing NVENC/VAAPI/QSV. `latest-nvidia` is not stale-but-current, it is abandoned – pinning to it silently freezes you on June 2025.

SEC.02

THE WEATHER DESK

`tools/z0-weather.sh` fetches from **Open-Meteo** (no key, no account, so nothing to rotate or expire), renders with the `ffmpeg` inside the ErsatzTV image, and installs each asset atomically. `tools/z0-weather.py` does the fetching and lays out the `drawtext` calls; the shell script drives `ffmpeg`.

BASH

```
tools/z0-weather.sh          # card + crawl + full-screen segment
tools/z0-weather.sh --card-only # skip the segment (the slow part)
```

Location comes from the environment, defaulting to Kelowna, BC:

BASH

```
Z0_WX_LAT=49.88307 Z0_WX_LON=-119.48568 Z0_WX_CITY=KELOWNA Z0_WX_REGION=BC \
Z0_WX_TZ=America/Vancouver tools/z0-weather.sh
```

Two cron entries on the playout host, because the two halves have very different costs – the card is seconds, the segment is most of a minute of encoding that competes with the live transcode:

```
* /15 * * * * tools/z0-weather.sh --card-only
5  * * * * tools/z0-weather.sh
```

A MUSIC BED

The segment ships with a quiet low drone. Drop any audio file into `weather/bed/` and the next render uses it instead, looped and faded. `Z0_SILENT=1` gives silence.

SEC.03

THINGS THAT WILL BITE YOU

Write atomically or the overlay disappears for a whole programme. ErsatzTV opens these files while streaming. If it catches a half-written PNG, `ImageElement` logs `Failed to initialize image element; will disable for this content` and the graphic is gone until the next item – a silent, self-healing failure that looks like a config mistake. Every install here is `write-temp + rename(2)`.

The segment's duration must never change. ErsatzTV scheduled it from the duration it saw at scan time. The script pins the output to `Z0_WX_SEGMENT_SECS` and **refuses to install** a file of the wrong length, so the library row stays valid and no rescan is ever needed. Keep one filename.

Overlays refresh per programme, not per frame. ErsatzTV re-reads an image or text template when each item's ffmpeg process starts. During a 90-minute film the card is whatever was on disk when that film began – which is why the card carries an `AS OF` stamp instead of pretending to be live. The segmenter also works ahead, so what is on air is a little behind the file on disk.

Skia does not use fontconfig. Text elements render through `SkiaSharp/RichTextKit`, which ignores `fontconfig` entirely – `fc-list` and `fc-match` finding a face inside the container proves nothing. Without `include_fonts_from` the log says `Could not find font ...; using default` and the lower third quietly renders in the wrong face. The two faces we need are staged in `branding/fonts/`.

drawtext is missing from some ErsatzTV images. The old 25.2 `-nvidia` image was

built without it despite advertising freetype and fontconfig, so the `make-*.sh` generators could not run against it. The 26.7.1 image has it.

A crawl over a feature film is what makes people switch off. The schedule turns the strip off around each feature and overnight during colour bars, and turns the weather card off during the weather segment itself (it repeats the segment and collides with its header).

A subtitle element costs a full-frame composite on every frame, and the price is the channel's resolution. This is the big one: it took the channel off air 25 times in one morning.

ErsatzTV composites graphics itself, in-process, and pipes one full-frame RGBA stream into ffmpeg as input `[1:0]`. Image and text elements are rasterised once and reused, so they are nearly free – the heaviest item on the channel ran **1.28x** with the bug, card and up-next all on. Add the subtitle element and the same class of item drops to **0.25x**, whatever the source: a 640x480 cartoon and a 1080p feature both landed there. ErsatzTV then logs

```
[FTL] Media item [N] on channel 0 transcoded at 0.254x (NOT throttled)
      which is NOT fast enough to support playback
```

abandons the item and cuts to **fallback colour bars, which carry no graphics at all** – so the symptom is "the overlays vanished", not "the channel is slow", and the guide, the DB and `/api/status` all still look perfect.

The clean natural experiment: item 2513 carried `{up-next, bug, strip}` and ran 0.258x; item 2291 carried `{up-next, card, bug}` and ran 1.28x. **Same number of elements** – so it is the subtitle element specifically, not the overlay count.

Three things it is **not**, each measured rather than assumed:

- ▶ **Not libass.** ffmpeg renders the exact same `.ass` at **635 fps** (21x realtime). The subtitle rasteriser was never the bottleneck.
- ▶ **Not the box.** 12 cores at load 2.2, no cgroup CPU limit, ErsatzTV pinned at ~238% – it had headroom and still underran. The cost is serialised inside one process, so more cores will not fix it.
- ▶ **Not animation.** The strip was rewritten from a scrolling `\move` crawl to static paged text, so every frame within a page is byte-identical. It made **no differ-**

ence at all – still 0.254x. ErsatzTV re-renders a subtitle element every frame whether or not anything changed, so there is no static-content fast path to exploit.

What actually fixes it is **resolution**, because the per-frame cost is proportional to pixel count. The channel went 1920x1080 → 854x480 → **640x480** (6.75x fewer pixels than 1080p) and the strip became affordable. That is the whole reason this is an SD channel.

640x480 is also the honest shape for this library: the cartoons and the Prelinger reels are natively 4:3, so they now fill the frame and the ffmpeg command carries **no pad= filter at all**. The cost is that genuine 16:9 items get letterboxed instead.

If you ever put the channel back up to 720p or 1080p, **the bottom strip has to go**, or the channel will sit on colour bars.

854x480 killed the 0.25x failures but left almost no margin – the Prelinger block still logged the odd **[FTL]** at **0.996x**. 640x480 took the same content to **1.33x**.

That number is worth keeping, because it is exactly the pixel ratio (854x480 / 640x480 = 1.334). **The reported speed tracks real capability, and this pipeline is purely pixel-throughput-bound**. I had first written those 0.996x fatals off as jitter against the **-readrate 1.0** cap ErsatzTV applies to playout items; the clean 1.33x scaling disproves that, since a hard cap at 1.0 could not be exceeded. They were real, and the margin at 854x480 really was about zero.

So: speed reports mean what they say. Treat anything under ~1.2x as a channel that will fall over eventually, and confirm with whether **fallback filler actually starts** – **grep -ci colorbars** over **/config/logs/ersatztv<date>.log**.

SEC.04

CHANGING THE CHANNEL'S RESOLUTION

Three things have to move together, and missing any one of them looks like a rendering bug rather than a config mistake:

1. **FFmpegProfile 1** – ResolutionId and VideoBitrate. There was no 16:9 480p row, so 854x480 was added as Resolution id 5; 640x480 is the stock id 0.
2. **PlayResX/PlayResY** in the strip's ASS (Z0_PLAYRES_X/Z0_PLAYRES_Y, default 1440x1080). libass scales x by frame_width/PlayResX and y by frame_height/PlayResY, so a 16:9 script rendered into the 4:3 channel squashes every glyph horizontally by a third. Keep PlayResY at 1080 and change only PlayResX with the aspect, and the font sizes and CRAWL_Y in the generator stay meaningful.
3. **The width-based percentages** in z0-bug.yml (scale_width_percent), z0-weather-card.yml (scale_width_percent) and z0-upnext.yml (width_percent). These are fractions of frame width, while the strip is keyed to frame height. Going 854->640 wide at the same height, the percentages had to rise by 854/640 (9→12, 22→29, 55→73) or those three would have shrunk by a quarter while the strip stayed put.

Then restart z0-ersatztv, then z0-uplink – Owncast is single-quality passthrough and cannot carry a mid-stream resolution change, so the tower drops for ~15 s and the uplink reconnects itself.

SEC.05

WIRING IT INTO THE SCHEDULE

tools/wire-graphics.py rewrites playout/channel-z0.yml, inserting the graphics_on/graphics_off instructions and the four daily forecast slots across all seven day blocks. It is idempotent – re-run it after editing the schedule.

BASH

```
tools/wire-graphics.py playout/channel-z0.yml playout/channel-z0.yml
```

It also repairs two things that stop the playout building on 26.x, both of which fail **silently**:

- ▶ **search: must be null.** This file used to write `search: Cartoons`, using the slot as a label. From 25.4 the schedule is validated against a JSON schema where `search` is `{"type": "null"}` and every object is `additionalProperties: false`. A failed validation means `SequentialPlayoutBuilder` returns `None` and the playout does not build at all. The label now lives in a comment, where it always belonged.
- ▶ **Every `pad_until` needs an explicit `tomorrow`:** 26.x changed `Tomorrow` from `bool` to `string` so it can hold an expression, and `YamlPlayoutPadUntilHandler` does `new Expression(padUntil.Tomorrow)` with no null check. `NCalc` throws `ExpressionString cannot be null or empty` and the whole build dies. The branch only runs when the pad target is already in the past, so a bare `pad_until` is a landmine that detonates the first time the playout is rebuilt later in the day than its earliest morning block.

After changing the schedule, the playout must be **reset** – graphics elements are attached to playout **items** as they are built, so existing items never gain them. Delete `PlayoutItem`, `PlayoutAnchor` and `PlayoutHistory` for the playout and restart. Reset on a **Wednesday** (see the header of `channel-z0.yml`).

Element definitions must exist and be **registered** before the build that references them. `ErsatzTV` refreshes them at startup, hourly, and when a stream session starts – but at startup it builds playouts **before** refreshing, so a brand-new element file needs one restart to register and a second build to be attached.

SEC.06

PERFORMANCE

Compositing is CPU work (SkiaSharp), and the blend is a GPU `overlay_cuda`. Adding four elements on top of a software x264 encode pushed some items below real time (one hit **0.253x**, which is a stutter, not a warning). The channel now encodes with **NVENC** (`HardwareAcceleration=2` on the `ffmpeg` profile) – the GTX 1650 was sitting completely idle while the CPU did all of it.

One title still runs near the line: an interlaced 480p print that gets deinterlaced and upscaled 3× (~1.26x). That is the source, not the overlays.

NVENC was necessary but **not sufficient** – it fixed the encode, not the overlay generation, and the channel still fell to colour bars once the bottom strip was on. That cost sits in ErsatzTV's own compositor, upstream of the encoder, so no encoder setting touches it. See "A subtitle element costs a full-frame composite on every frame" above.

The channel therefore runs at **640x480 / 1200k** rather than 1920x1080 / 2000k. Rule of thumb: image and text elements are free, a subtitle element costs about 4x realtime at 1080p, and that price scales with pixel count.

SOURCE docs/weather.md SHA 55a0ccdc1efb BUILT 2026-08-13